

Integrity from Algebraic Manipulation Detection in Trusted-Repeater QKD Networks

Ailsa Robertson^{a,1,2} , Christian Schaffner^{b,1,2}  and
Sebastian R. Verschoor^{c,1,2} 

¹ University of Amsterdam, The Netherlands

² QuSoft, The Netherlands

Abstract. Quantum Key Distribution (QKD) allows secure communication without relying on computational assumptions, but can currently only be deployed over relatively short distances due to hardware constraints. To extend QKD over long distances, networks of trusted repeater nodes can be used, wherein QKD is executed between neighbouring nodes and messages between non-neighbouring nodes are forwarded using a relay protocol. Although these networks are being deployed worldwide, no protocol exists which provides provable guarantees of integrity against manipulation from both external adversaries and corrupted intermediates. In this work, we present the first protocol that provably provides both confidentiality and integrity. Our protocol combines an existing cryptographic technique, Algebraic Manipulation Detection (AMD) codes, with multi-path relaying over trusted repeater networks. This protocol achieves Information-Theoretic Security (ITS) against the detection of manipulation, which we prove formally through a sequence of games.

Keywords: QKD · quantum key distribution · secret sharing · authentication codes · trusted repeater networks · information theoretic security · provable security

1 Introduction

Parties that can exchange quantum information can use Quantum Key Distribution (QKD) to expand a small shared secret into an arbitrarily long secret [BB84]. The effectiveness of such quantum channels, both in terms of distance and rate of transmission, is currently restricted by hardware limitations. While *quantum repeaters* are able to lift this restriction and establish long-distance end-to-end quantum channels, their realisation is currently restricted to laboratory demonstrations and cannot yet be deployed at scale in a quantum network.

An alternative method for establishing keys over a long distance is to use trusted repeater networks [SPD⁺10]. In such networks, QKD is executed between neighbouring nodes of the network, and messages between non-neighbouring nodes are then established via a *relay* protocol. In the simplest relay protocol, intermediate neighbours decrypt and re-encrypt: if Alice wants to send a message to Bob, she can encrypt and send it to Ingrid (an intermediate node) using a QKD-generated key. Ingrid can decrypt to recover the message and then re-encrypt and send it to Bob. If more intermediate nodes are required

E-mail: s.r.verschoor@uva.nl (Sebastian R. Verschoor)

^aFunded by the Dutch National Science Agenda (Nationale Wetenschapsagenda, NWA) by The Dutch Research Council (NWO) under project number 1436.20.002, Quantum Impact on Societal Security.

^bPartially supported by gravitation project Challenges in Cyber Security (CiCS) with file number 024.006.037 which is financed by the NWO.

^cSupported by the Dutch National Growth Fund (NGF), as part of the Quantum Delta NL programme.



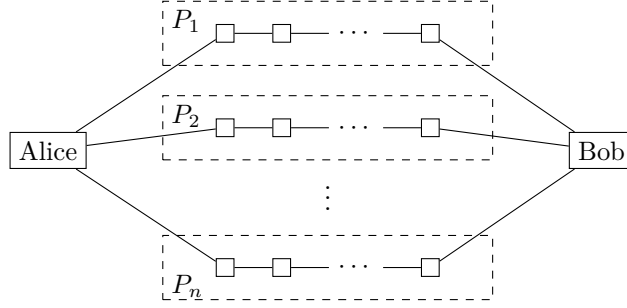


Figure 1: Alice is connected to Bob via n disjoint paths. Each path P_i consists of at most ℓ nodes, and no node may appear in more than one path.

to reach the distance, each node can act as Ingrid and re-encrypt the key towards the next node. Relaying does not provide end-to-end security, since intermediate nodes must be trusted to forget and securely delete the secret information after relaying.

To reduce the level of trust required of intermediate nodes, parties can use a multi-path relaying protocol. Assume for example that Alice and Bob do not share a direct QKD link, but both have a QKD link with both Ingrid and Inez. Alice generates two random secrets S_1 and S_2 , and relays both to Bob: she relays S_1 via Ingrid and S_2 via Inez. Both Alice and Bob then output $S_1 \oplus S_2$ as their shared secret. In order to learn the secret output, the adversary would have to corrupt both Ingrid and Inez.

This generalises to a network with n disjoint paths, each consisting of one or more intermediate nodes, shown in Figure 1. Alice splits her secret into n shares using a secret-sharing scheme and relays one share per path from which Bob can recover the secret.

Our contributions Note that if Bob’s output differs from Alice’s, then at least one share or message must have been tampered with. In this case, it is desirable that Bob can detect tampering and reject the protocol run. This property is called integrity; we claim that no existing solutions provide *provable* integrity against such manipulation. In this work, we present the first protocol which provides provable integrity against manipulation by computationally unbounded in trusted-repeater QKD networks. Our protocol, listed in Listing 1 and visualised in Figure 3, is conceptually simple. First, Alice encodes her secret using an existing construction from AMD codes (see Section 2.2). She then uses secret sharing (see Section 2.1) and finally she relays each share over its own path, using the QKD generated keys as a One-Time Pad (OTP). As we aim for ITS, meaning that security must hold against computationally unbounded adversaries, we assume an upper bound on the number of paths that the adversary can corrupt. We formally prove security by adapting the existing game-based model [BR07] to the context of trusted repeater networks, and we show that the overhead (in number of additional QKD bits consumed) is optimal, and we provide an implementation to demonstrate our methods are efficient.

1.1 Related Work

Various solutions have been proposed which aim to achieve integrity in the trusted repeater context, most notably the SECOQC protocol [SPD⁺10]. We summarise these solutions and address the issues we found.

1.1.1 SECOQC

As part of the SECOQC project, Salvail, Peev, Diamanti, Alléaume, Lütkenhaus and Länger [SPD⁺10] analysed the security of trusted repeater networks. They proposed to

use a secret-sharing scheme to exchange a secret confidentially, followed by an interactive protocol to achieve integrity. In their protocol, Alice locally computes random parity checks over part of the secret-sharing scheme secret and subsequently uses the remaining bits of the secret-sharing scheme output as a Message Authentication Code (MAC) key to authenticate the transmission of the parity check equations. The authors claim that their protocol enables Bob not only to *detect* cheating but also to *identify* the cheater.

The issue with this approach is that MACs only guarantee security under the assumption that the communicating parties already share an identical key. This constitutes circular reasoning and thus the protocol does not yield provable security. While we do not provide an explicit attack against cheater detection, we do demonstrate in [Section A](#) that this allows an adversary to break the cheater identification of the protocol. The attack performs algebraic manipulation to the MAC key, causing Bob to misidentify the malicious parties.

1.1.2 Other solutions

Beals, Hynes and Sanders proposed the *stranger authentication protocol*: an interactive protocol for checking the equality of a secret-sharing scheme output which does not assume pre-shared keys [BS08; BHS09]. In their protocol, Alice and Bob have secret-sharing scheme outputs $S = S_1 \| S_2 \| S_3$ and $S' = S'_1 \| S'_2 \| S'_3$, respectively. Alice generates a random number r and transmits $(r, H(S_3)) \oplus S_1$, and Bob replies with $H(r) \oplus S'_2$. The function H is described as a hash function, but its requirements are only specified informally and it is unclear whether they can be satisfied against a computationally unbounded adversary. The authors remark that their protocol assumes S and S' are unknown to the adversary, whereas we observe that our method ensures integrity even if the adversary can choose S .

Another notable solution is presented by Rass and König [RK12]. While the primary focus of their work is different from ours as they address security against adversaries capable of rerouting shares across the network, the aspect relevant to our work is their method for detecting share manipulation. They suggest that an “obvious quickfix” is to attach a checksum to the dealt secret. Their proposed protocol employs a one-time MAC tag as a checksum, however the authors neither specify a MAC key nor clarify what an appropriate key would be. They assume no pre-shared keys, and we observe that deriving the key from the dealt secret would lead to the same problematic circular reasoning observed in [Section 1.1.1](#).

Outline In [Section 2](#), we provide the necessary background for this work. In [Section 3](#), we provide our protocol ([Listing 1](#)) which uses an existing construction from cryptographic literature. We argue that the protocol achieves the desired security properties efficiently. We conclude with future directions in [Section 4](#).

2 Background

In this section, we provide the necessary background on secret sharing, AMD codes and trusted repeater networks. We will use the symbol $\perp$ to indicate a missing value, with addition defined such that $\perp + x = x + \perp = \perp$ for all x .

2.1 Secret Sharing

A secret-sharing scheme allows a *dealer* to split their secret into n shares, such that the secret can be completely recovered from a *qualified* subset of shares, but an *unqualified* subset of shares reveals no information about the secret [Sha79]. The subsets that are qualified are defined by an *access structure*, which is a property of the specific scheme in use. We adapt the definition of a secret-sharing scheme from other literature [CDF⁺08].

Definition 1 (Secret-Sharing Scheme). A secret-sharing scheme is a pair of functions ($\text{Share}, \text{Recover}$). Let $\mathcal{S}$ denote the secret space and $\mathcal{Y}^n$ the space of n shares. Given a secret $S \in \mathcal{S}$, the probabilistic function Share splits S into a vector $\vec{S} = (S_1, \dots, S_n) \in \mathcal{Y}^n$, called the *shares* of S . The function Recover recovers $S' \in \mathcal{S} \cup \{\perp\}$ from a vector of shares $\vec{S}' = (S'_1, \dots, S'_n) \in (\mathcal{Y} \cup \{\perp\})^n$. Shares may be omitted, in which case $S'_i = \perp$ for some index i , and recovery may fail or reject, in which case $S' = \perp$.

Let $B \subseteq \{1, \dots, n\}$ be any *qualified* set. Then correctness requires that

$$\text{Recover}(\vec{S}') = S \quad \text{for} \quad S'_i = \begin{cases} S_i & i \in B \\ \perp & i \notin B. \end{cases}$$

Perfect confidentiality requires that for any *unqualified* subset $A \subset \{1, \dots, n\}$, the shares $\{S_i\}_{i \in A}$ reveal no information about S .

While the confidentiality requirement is presented somewhat informally in the above definition, it has a natural formalisation through the concept of indistinguishability [BR07].

One of the simplest secret-sharing schemes is additive secret sharing, which distributes a secret as a sum of random shares. In an additive group such that $S \in \mathcal{G}$, $\text{Share}(S)$ generates $(n - 1)$ shares as uniformly random group elements $S_i \xleftarrow{\$} \mathcal{G}$ for $1 \leq i < n$ and sets the last share to value $S_n = S - \sum_{i=1}^{n-1} S_i$. Recover simply adds the shares: $S' = \sum S'_i$. One important property of this scheme is that it is linear, meaning that $\text{Recover}(\vec{S} + \vec{S}') = \text{Recover}(\vec{S}) + \text{Recover}(\vec{S}')$, where vector addition is defined element-wise.¹ In the additive secret sharing scheme, only the complete set of shares constitutes a qualified set. A notable special case operates over the group of bitstrings under bitwise addition modulo 2, i.e. the bitwise exclusive-or ($\oplus$) operation.

Our method can easily be generalised to other linear secret-sharing schemes with more flexible access structures, such as Shamir’s scheme [Sha79].

2.1.1 Robust secret sharing

Robust secret sharing protects against an adversary that aims to alter the recovery output ($S' \notin \{S, \perp\}$) by changing one or more of the shares ($S'_i \notin \{S_i, \perp\}$). Many definitions of robust secret sharing exist in the literature [MS81; TW89; OK96; BR07], expressing subtly different security guarantees. Some schemes merely want to *detect* any tampering, while others want to be able to *correct*, i.e. recover $S' = S$ even when some $S'_i \neq S_i$. Some schemes also aim to *identify* whom the cheating shareholders are. *Strong* robustness protects when sharing any secret S , while *weak* robustness assumes that S was sampled uniformly random and is unknown to the adversary [CDF⁺08]. Some schemes assume that recovery is done by an honest shareholder (who knows that at least their own share is correct), while others consider recovery by an external party. For concrete security, the distinction between *dynamic* and *static* corruption is important [BR07].

In general, robust secret sharing adds some redundancy to the shares, increasing their size beyond that of the shared secret. A strongly robust scheme will require $2 \log 1/\delta$ bits to detect tampering with probability at least δ [CDF⁺08].

The scheme we consider achieves detection, no identification, external recovery, and (a variant of) strong robustness, all in the context of dynamic corruption. We require a variant of robustness where the adversary can choose some shares and induce a shift on uncorrupted shares, which we introduce in Section 3, Definition 5. In Section B.2, we adapt an existing game-based formalisation of robust secret sharing accordingly [BR07].

¹Formally, linearity holds only with respect to non-missing shares, which is sufficient for our purposes.

2.1.2 Weak robust secret sharing

Secret sharing can be used directly on secret messages, or they can be used to establish a shared key between Alice and Bob. In the latter case, weak robust secret sharing suffices, which can provide the same security guarantees with smaller overhead in the size of the shares. Weak δ -robustness was found to require growing the shares by $\log 1/\delta$ bits [CDV94], an improvement of a factor two over strong robustness. External difference families lead to weakly secure AMD codes with minimal tag size [CDF⁺08].

We summarise the two strands of literature in which weak robust secret sharing is implemented using *difference sets*. First, attempts had previously been made to implement Shamir's secret-sharing scheme in a way that protects against malicious shareholders, using elements of a field F [TW89] and a small subset of legitimate values $S \subset F$. Ogata and Kurosawa observed that the size of S can be reduced by the use of difference sets [OK96]. Ogata and Kurosawa proved that their construction is optimal for the size of shares and that the difference sets defined by Singer [Sin38] suffice for this construction. They also find useful parameter restrictions, which are that both q and $q^2 + q + 1$ must be prime. We note that this scheme is proven secure. Although the scheme is optimal in terms of share size, it is not certain that there exists an efficient implementation of this scheme. Specifically, we note that while it might be efficient to sample random points on a line in finite projective plane $PG(2, q)$ (as in Singer's work), finding the corresponding value i in the difference set involves solving a discrete logarithm. The search for an efficient implementation of the scheme remains open.

Our protocol does not employ weak robust secret sharing, but we note that it is a possible avenue for improvement when sharing keys instead of arbitrary messages. In Section 4, we will reflect on the use of external difference families for weak robust secret sharing in trusted repeater QKD networks.

2.2 Algebraic Manipulation Detection

In 2008, Cramer, Dodis, Fehr, Padró and Wichs provided an encoding of data that provides Algebraic Manipulation Detection [CDF⁺08]. Their code maps values to a group in such a way that any nontrivial shift of an encoded value will be detected with high probability by the decoder.

Definition 2 (Algebraic Manipulation Detection (AMD) code [CDF⁺08]). A δ -AMD code is a pair of functions (Encode, Decode): a probabilistic encoding map $\text{Encode} : \mathcal{S} \rightarrow \mathcal{G}$, from some set $\mathcal{S}$ to a group $\mathcal{G}$, and a decoding map $\text{Decode} : \mathcal{G} \rightarrow \mathcal{S} \cup \{\perp\}$ such that $\text{Decode}(\text{Encode}(s)) = s$ with probability 1 for all $s \in \mathcal{S}$. The security requirement is that for any $s \in \mathcal{S}$ and any $\Delta \in \mathcal{G}$ it holds that

$$\Pr[\text{Decode}(\text{Encode}(s) + \Delta) \notin \{s, \perp\}] \leq \delta.$$

The authors also present the following construction and prove its security.

Construction 1 ([CDF⁺08]). Let $\mathbb{F}$ be a field of size q and characteristic p , and let d be an integer such that $d + 2$ is not divisible by p . To encode a value $s = (s_1, \dots, s_n) \in \mathbb{F}^d$, define $\text{Encode} : \mathbb{F}^d \rightarrow \mathbb{F}^d \times \mathbb{F} \times \mathbb{F}$ given by $\text{Encode}(s) := (s, x, f(x, s))$, where $x \xleftarrow{\$} \mathbb{F}$ is sampled uniformly random, and $f(x, s) := x^{d+2} + \sum_{i=1}^d s_i x^i$. Decoding is naturally given by

$$\text{Decode}(s', x', \sigma') = \begin{cases} s' & \text{if } f(x', s') = \sigma' \\ \perp & \text{otherwise.} \end{cases}$$

The authors present several applications that make use of the AMD code, including the robust secret-sharing scheme (Share^* , Recover^*) defined below.

Construction 2 ([CDF⁺08]). Let $(\text{Share}, \text{Recover})$ be a linear secret-sharing scheme and let $(\text{Encode}, \text{Decode})$ be an AMD code, then the robust secret-sharing scheme $(\text{Share}^*, \text{Recover}^*)$ is defined by $\text{Share}^*(s) = \text{Share}(\text{Encode}(s))$ and $\text{Recover}^*(\vec{S}'_i) = \text{Decode}(\text{Recover}(\vec{S}'_i))$.

The authors prove that **Construction 2** is a strongly robust secret-sharing scheme, formalising robustness as the detection of tampering by an honest shareholder.

2.3 Trusted Repeater Networks

A trusted repeater network is a graph $G = (V, E)$ where the vertices $v \in V$ (or nodes) correspond to the protocol participants, and edges $(v, v') \in E$ indicate which parties share a key. Alice and Bob are nodes in the network; other nodes represent trusted repeaters. A path $P_i \subseteq E$ between two nodes is a sequence of edges that connect them. Two paths are disjoint if none of their edges connect to the same vertex, except for the first and last vertex. We will only consider paths from Alice to Bob that are pairwise disjoint, which allows for the following notation (see also **Figure 2**). We will provide values with two indices: the first index i refers to the i -th path, while the second index j refers to the j -th edge on that path. For example, Alice shares a key q_{i1} with her neighbour on path P_i .

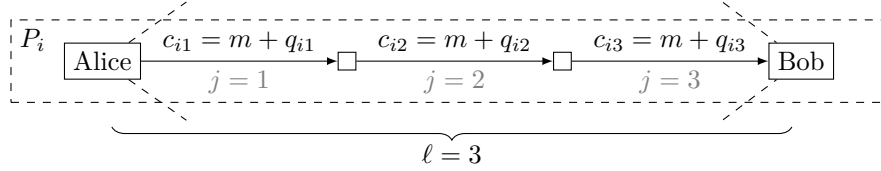


Figure 2: Alice relays message m to Bob on a single path P_i of length $\ell = 3$. Encryption on the j -th edge is done with an OTP using key q_{ij} , resulting in ciphertext c_{ij} .

We consider only a simple relaying protocol, in which each node receives an encrypted message from a neighbouring node, then decrypts the message and re-encrypts it before forwarding it to the next node along the designated path. We will consider OTP encryption in some additive group. For example, in **Figure 2**, the first trusted repeater will receive c_{i1} from Alice, and will relay $c_{i2} = c_{i1} - q_{i1} + q_{i2}$ to the next trusted repeater.

If the adversary corrupts a node, they obtain all secrets held by that node. A node is considered honest if it has not been corrupted; Alice and Bob are assumed to be honest parties. Honest parties securely delete keys after use. Nodes that are not directly connected (i.e. non-neighbouring nodes) do not share pre-established keys. All communication between nodes is observable by the adversary, who has full control over the network, which includes altering, dropping, inserting, misdelivering, reordering and replaying messages.

Multipath protocols decrease the level of trust required of intermediate nodes. The idea is as follows: assume there are n pairwise disjoint paths from Alice to Bob. Alice acts as the dealer and splits her secret in n shares, then uses relaying to transmit each share over its own path so that Bob can recover the secret when he receives all shares. The protocol provides confidentiality against any unqualified adversary, defined as any adversary whose corruption of nodes does not reveal a qualified set of shares, which is in turn defined by the secret sharing scheme. For the additive sharing scheme, this means any adversary that leaves all nodes uncorrupted on at least one path. We present a multipath protocol that also provides integrity against any unqualified adversary.

2.3.1 Single QKD Links

While our results apply to trusted repeater networks in general, our focus is on QKD networks, where current technological limitations prevent end-to-end quantum channels,

and trusted repeater networks are being deployed to overcome these distance limits. In this context, the keys shared between neighbours are QKD-generated keys. In order to capture the scenario where an adversary can gain quantum side information (information about the quantum system via, for instance, entangled states), we use the security definition by Renner [Ren05, Criterion (2.6)].

Definition 3 (statistical privacy against quantum side information). A classical-quantum state

$$\rho_{XE} = \sum_x P_X(x) |x\rangle\langle x| \otimes \rho_E^x,$$

is said to be ε -private against quantum side information if

$$\frac{1}{2} \|\rho_{XE} - \rho_U \otimes \rho_E\|_1 \leq \varepsilon, \quad (1)$$

where ρ_{XE} is the joint state of the classical variable X and the adversary's quantum system E , $\rho_U = \sum_x \frac{1}{|\mathcal{X}|} |x\rangle\langle x|$ is the uniform distribution over the support $\mathcal{X}$ of X , $\rho_E = \sum_x P_X(x) \rho_E^x$ is the reduced state of the adversary's system and $\|\cdot\|_1$ denotes the trace norm.

A distribution that is statistically ε -private is statistically indistinguishable from the uniform distribution by any unbounded adversary.

In QKD, the adversary is usually given complete control over the quantum channel. Therefore, the adversary can always cause the QKD protocol to abort and not produce a key. In our scenario, repeated aborts between two neighbouring QKD nodes v, v' in the network would result in the loss of the edge (v, v') in the trusted repeater network. One could model this power of the adversary by considering networks with probabilistic or adversarial edges. However, for simplicity, we assume a *static* network topology. In other words, we assume for every edge $(v, v') \in E$, the adversary does not induce a protocol abort. Similarly, a QKD protocol run could be incorrect, i.e. adversary interference or inherent protocol imperfections could cause the generated key to be different for the two parties, therefore rendering this link invalid. Again, we assume for simplicity that this does not occur for any edge $(v, v') \in E$. Therefore, the imperfection of a *single* QKD link is captured by Definition 3 above.

2.3.2 QKD Networks

While Definition 3 above captures secrecy of a single QKD link, our scenario requires a network of QKD links. In this scenario, we consider a central adversary with control over all (quantum) communication in the network. We therefore generalise Definition 3 in a natural way.

Definition 4 (collective statistical privacy against quantum side information). For k classical variables $X^k = (X_1, X_2, \dots, X_k)$, we call the state

$$\rho_{X^k E} = \sum_{x^k} P_{X^k}(x^k) |x^k\rangle\langle x^k| \otimes \rho_E^{x^k},$$

ε -private against quantum side information if

$$\frac{1}{2} \|\rho_{X^k E} - \rho_U \otimes \rho_E\|_1 \leq \varepsilon, \quad (2)$$

where $\rho_{X^k E}$ is the joint state of the classical variables X^k and the adversary's quantum system E , $\rho_U = \sum_{x^k} \frac{1}{|\mathcal{X}^k|} |x^k\rangle\langle x^k|$ is the uniform distribution over the support $\mathcal{X}^k$ of X^k , $\rho_E = \sum_{x^k} P_{X^k}(x^k) \rho_E^{x^k}$ is the reduced state of the adversary's system and $\|\cdot\|_1$ denotes the trace norm.

Lemma 1. *In a QKD network with k individual QKD links that are ε -private according to Definition 3, the overall state is $k\varepsilon$ -private according to Definition 4.*

Proof sketch. Intuitively, the claim follows from the universal composability of QKD keys. Concretely, the randomness of the keys generated in the individual QKD runs is generated by honest players, and hence the keys are independent of each other. Therefore, a global eavesdropper does not have the ability to correlate the generated keys. The claimed overall privacy bound of $k\varepsilon$ then follows by a union bound from the ε -privacy of the individual keys. $\square$

3 Using AMD Codes in Trusted Repeater Networks

Our protocol is straightforward: Alice shares a secret using Construction 2 and relays the shares over the trusted repeater network. The protocol is defined in Listing 1 and visualised in Figure 3. We use notation c_{ij} for the ciphertext message sent on the j -th edge of the i -th path, c'_{ij} for the delivered ciphertext (possibly changed by the adversary) and q_{ij} for the corresponding key.

Listing 1 Protocol for achieving confidentiality and integrity in trusted relay networks.

```

1: procedure ALICE( $s$ )
2:    $\vec{S} \leftarrow \text{Share}^*(s)$                                  $\triangleright \vec{S} = (S_1, \dots, S_n) = \text{Share}(\text{Encode}(s))$ 
3:   for  $1 \leq i \leq n$  do
4:      $c'_i \leftarrow S_i + q_{i1}$                                  $\triangleright \text{Encrypt. } \vec{c} = (c_{11}, \dots, c_{n1})$ 
5:     delete  $q_{i1}$ 
6:   return  $\vec{c}$                                                $\triangleright \text{Ciphertext } c_i \text{ is sent to path } P_i$ 
7: procedure RELAY $_{ij}(c'_{i(j-1)})$    $\triangleright \text{The } j\text{-th node on path } i \text{ receives ciphertext } c'_{i(j-1)}$ 
8:    $c_{ij} \leftarrow c'_{i(j-1)} - q_{i(j-1)} + q_{ij}$                $\triangleright \text{Re-encrypt}$ 
9:   delete  $q_{i(j-1)}$  and  $q_{ij}$ 
10:  return  $c_{ij}$                                               $\triangleright \text{Ciphertext } c_{ij} \text{ is sent to the } (j+1)\text{-th node on path } i$ 
11: procedure BOB( $\vec{c}'$ )                                 $\triangleright \text{Ciphertext } c'_i \text{ is received from path } P_i$ 
12:  for  $1 \leq i \leq n$  do
13:     $S'_i \leftarrow c'_{i\ell} - q_{i\ell}$                              $\triangleright \text{Decrypt. } \vec{S}' = (S'_1, \dots, S'_n)$ 
14:    delete  $q_{i\ell}$ 
15:   $s' \leftarrow \text{Recover}^*(\vec{S}')$                          $\triangleright \text{Bob outputs } s' = \text{Decode}(\text{Recover}(\vec{S}'))$ 

```

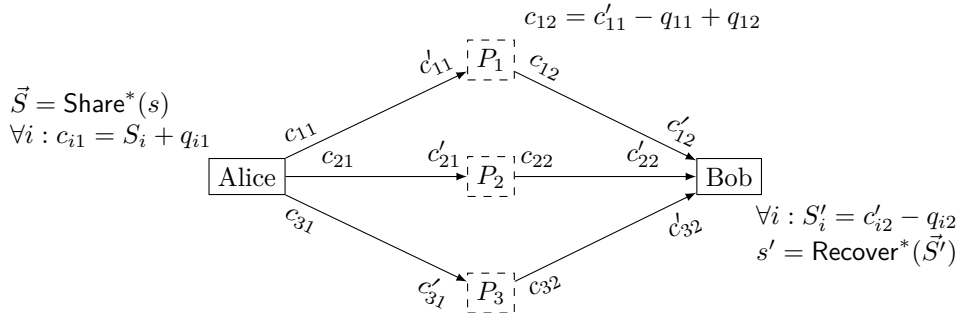


Figure 3: Protocol for transmitting a secret s over the encrypted channels of the trusted repeater network, visualised here for $n = 3$ and $\ell = 2$. Alice and Bob output s and s' , respectively, where we note that $s' = \perp$ if decoding fails, indicating that Bob rejects.

3.1 Security analysis

For this work we make strong assumptions on the trusted repeater network: the network G is fixed, the endpoints are fixed, and even the keys used between neighbours are fixed. We reflect on these assumptions in [Section 3.2.2](#).

We first consider the confidentiality of the protocol; this reasoning is formalised in [Section B.1](#).

Theorem 1 (Confidentiality (informal)). *Fix a trusted repeater network with n disjoint paths, each of length at most ℓ , where all neighbours share a key that is ε -private against quantum side information. Let $(\text{Share}, \text{Recover})$ be a perfectly confidential secret-sharing scheme. Then the protocol of [Listing 1](#) provides $n\ell\varepsilon$ -confidentiality against all unqualified adversaries.*

Proof sketch. Each encryption with an ε -private key leaks at most ε information about the encrypted share. There are $n\ell$ total encryptions; thus, by [Lemma 1](#), the total information leaked is $n\ell\varepsilon$. Given our assumption that the adversary corrupts only an unqualified set of paths or shares, the perfect confidentiality of the secret-sharing scheme guarantees no further leakage. $\square$

We note that the network width n occurs in the above bound because the adversary may corrupt nodes dynamically. In the case of static corruption, the total leakage can be bounded by summing over only the fixed, uncorrupted path(s).

Next we consider integrity of the protocol. [Construction 2](#), $(\text{Share}^*, \text{Recover}^*)$, is known to be robust when a shareholder recovers the secret [[CDF⁺08](#), Theorem 3]. In contrast, recovery is external in the trusted repeater network setting: Bob cannot assume receipt of at least one honest share. Given that OTP encryption is malleable, the adversary can make a blind change to the honest shares. We therefore generalise robustness to account for this adversarial capability.

Definition 5 (Shift-robustness). A secret-sharing scheme $(\text{Share}, \text{Recover})$ is δ -shift-robust if for any secret s , and for any computationally unbounded adversary $\mathcal{A}$ that corrupts an unqualified set of paths $A \subset \{1, \dots, n\}$, we have the following: Let $\vec{S} = \text{Share}(s)$, let $a_i = \mathcal{A}(i, \{S_a\}_{a \in A}) \in \mathcal{G} \cup \{\perp\}$ be the i -th adversarially chosen value and let $\vec{S}'$ be a vector such that, for $1 \leq i \leq n$,

$$S'_i = \begin{cases} a_i & \text{if } i \in A \\ S_i + a_i & \text{if } i \notin A \end{cases}$$

then $\Pr[\text{Recover}(\vec{S}') \notin \{s, \perp\}] \leq \delta$.

We now prove that [Construction 2](#) is δ -shift robust. The proof is very similar to the proof for robust secret sharing [[CDF⁺08](#)].

Lemma 2. *Let $(\text{Encode}, \text{Decode})$ be a δ -AMD code, and $(\text{Share}, \text{Recover})$ be a perfect linear secret-sharing scheme code, then the secret-sharing scheme $(\text{Share}^*, \text{Recover}^*)$ defined in [Construction 2](#) is a δ -shift robust secret-sharing scheme.*

Proof. Let $S = \text{Share}^*(s)$ and let $\vec{S}'$ be a vector meeting the requirements of [Definition 5](#). Let $\vec{\Delta} = \vec{S}' - \vec{S}$. By the linearity of the secret-sharing scheme we have

$$\begin{aligned} \Pr[\text{Recover}^*(\vec{S}') \notin \{s, \perp\}] &= \Pr[\text{Decode}(\text{Recover}(\vec{S}') + \text{Recover}(\vec{\Delta})) \notin \{s, \perp\}] \\ &= \Pr[\text{Decode}(\text{Encode}(s) + \Delta) \notin \{s, \perp\}] \end{aligned}$$

where $\Delta = \text{Recover}(\vec{\Delta})$ is determined by the adversarial strategy. By the perfect confidentiality of the secret-sharing scheme, $\text{Encode}(s)$ is unknown to the adversary (except for

a-priori knowledge of s), so Δ is independent of the randomness used in $\text{Encode}(s)$. The lemma then follows immediately from the definition of AMD codes (Definition 2). $\square$

The protocol of Listing 1 provides integrity against any unqualified adversary, meaning they corrupted only an unqualified set of paths. We argue integrity informally below and formalise this reasoning later in Section B.2.

Theorem 2 (Integrity (informal)). *Fix a trusted repeater network with n disjoint paths, each of length at most ℓ , where all neighbours share a key that is ε -private against quantum side information. Let (Share, Recover) be a perfectly confidential and δ -shift robust secret-sharing scheme. Then the protocol of Listing 1 provides $(n\ell\varepsilon + \delta)$ -integrity against all unqualified adversaries.*

Proof sketch The proof proceeds through a sequence of games. In the first gamehop, we replace all QKD keys with uniform group elements; by Lemma 1, the resulting game is indistinguishable, except with probability at most $n\ell\varepsilon$. In the second gamehop, we reduce to the security of the shift-robust secret-sharing scheme. By Lemma 2, the winning probability of the last game is δ , so we add that term. We now outline the reduction for the second gamehop, assuming $n = 3$ and $\ell = 2$ as in Figure 3 for simplicity. The full version of this proof sketch in Section B.2 generalizes this to any n and ℓ .

Given any adversary $\mathcal{A}$ that breaks the integrity of the relay protocol with perfect keys, we construct an adversary $\mathcal{B}$ against the δ -shift robustness of the secret-sharing scheme by simulating the relay protocol of $\mathcal{A}$. We only consider one intermediate per path to avoid notational clutter; the formal proof extends to paths of arbitrary length.

Let $\vec{c} = (c_{11}, c_{21}, c_{31})$ denote the ciphertexts sent by Alice; $\mathcal{B}$ simulates these ciphertexts by sending a uniformly random vector. When $\mathcal{A}$ corrupts path P_i , $\mathcal{B}$ corrupts the i -th shareholder to get S_i and derives $q_{i1} = c_{i1} - S_i$. The other key is sampled uniformly random ($q_{i2} \xleftarrow{\$} \mathcal{G}$). When $\mathcal{A}$ queries a relay on a corrupted path, the relay returns the computed key. On uncorrupted paths, the relay returns a uniformly random ciphertext, which is equivalent to relaying with a random key q_{i2} . Finally, $\mathcal{A}$ outputs $\vec{c}' = (c'_{12}, c'_{22}, c'_{32})$ and from this $\mathcal{B}$ derives the vector $\vec{a} = (a_1, a_2, a_3)$ as follows: if a path is corrupted, then $\mathcal{B}$ can simply decrypt to obtain $a_i = c'_{i2} - q_{i2}$, as Bob would do in the relay protocol. For uncorrupted paths, $\mathcal{B}$ sets a_i to the sum of differences between received and sent ciphertexts:

$$a_i = c'_{i2} - c_{i2} + c'_{i1} - c_{i1}.$$

Recovery decrypts with q_{i2} to get $S'_i = c'_{i2} - q_{i2}$. From

$$\begin{aligned} c_{i1} &= S_i + q_{i1} \\ c_{i2} &= c'_{i1} - q_{i1} + q_{i2} \end{aligned}$$

we get the required value

$$\begin{aligned} S_i + a_i &= S_i + c'_{i2} - c_{i2} + c'_{i1} - c_{i1} \\ &= S_i + c'_{i2} - (c'_{i1} - q_{i1} + q_{i2}) + c'_{i1} - (S_i + q_{i1}) \\ &= c'_{i2} - q_{i2} = S'_i. \end{aligned}$$

Note that if no message was relayed on path P_i and thus c_{i2} and c'_{i1} are not defined when $\mathcal{A}$ outputs $\vec{c}'$, then $\mathcal{B}$ can sample a_i uniformly from random. This is equivalent to decryption with a uniformly random key. $\square$

3.2 Properties of Protocol

We outline the advantages of our protocol and examine the validity of notable assumptions.

3.2.1 Advantages

We highlight three advantages of the proposed protocol. First, we improve significantly on all existing solutions. Above all, our protocol is proven to be secure, unlike any other existing protocol. Additionally, we note that the SECOQC protocol [SPD⁺10] is an interactive protocol with $3n\ell$ authenticated messages between network neighbours, while in our protocol is non-interactive (Alice only sends and Bob only receives) and requires $n\ell$ non-authenticated messages between neighbours.

Second, our protocol yields an efficient implementation. For a field $\mathbb{F}$ of size q , the encoding of [Construction 1](#) maps a value $s \in \mathbb{F}^d$ to values in $\mathbb{F}^d \times \mathbb{F} \times \mathbb{F}$, and additive secret sharing has no overhead. For strong robustness (i.e. for relaying arbitrary messages), we achieve the optimal overhead of $2 \log 1/\delta$ bits [CDF⁺08]. To demonstrate the efficiency and simplicity, we provide a concrete implementation in [Section C](#).

Third, we note that our protocol works with any linear secret sharing scheme, allowing users to choose their trade-off between efficiency and security. If n -out-of- n secret sharing is desired, then the simple additive scheme can be used. If it is beneficial to allow recovery from fewer than n shares, for instance if network edges might run out of QKD key material, then Shamir's t -out-of- n scheme with $t < n$ may be a preferable choice of secret sharing scheme. It should be noted that reducing the size of qualified sets also reduces the number of paths an adversary would need to corrupt in order to recover the secret.

3.2.2 Assumptions

The first notable assumption relates to node authentication. In Authenticated Key Exchange (AKE) parties typically identify their peers by knowledge of some secret key: in asymmetric AKE this key is the private component of a public-key pair, while in symmetric AKE this is the shared key. In trusted repeater networks, however, such an end-to-end key is lacking. In trusted repeater networks, a node's identity can at best be determined by its location within the network. Consequently, authentication in such a network relies heavily on the trustworthiness of neighbouring nodes and their connections. In models where the adversary has some control over the selection of the endpoints and/or the relaying paths, authentication will depend on whether Bob can (cryptographically) verify if the claimed endpoint and paths are correct. By the strong assumption we made that the network is static, the assumption of node authentication is achieved. This is only reasonable if it can be ensured either by the physical properties of the network or by employing an additional (cryptographic) protocol that provides this guarantee in a dynamic network that is only partially trustworthy.

The second notable assumption is, as detailed in [Section 3.1](#), that elements of this model are fixed. This inability to represent dynamic scenarios is a significant limitation. In particular, it assumes that both parties and communication paths remain static, and that the keys used for relaying are known in advance, assumptions that restrict practicality severely. Real-world implementations may require greater flexibility, such as the ability to execute the protocol multiple times between different parties.

Finally, we reflect on the choice to add integrity on top of an existing network, which required the above assumptions about the network. Our protocol ensures that any modifications to messages travelling over this pre-authenticated network, whether by external parties or untrustworthy nodes, are detected, which we describe as integrity. Existing literature has referred to this concept as authenticity [CDF⁺08; BHS09] or robustness [RK12]. Although the term robustness is more consistent with the existing literature on secret-sharing schemes, we avoid using it due to its potential ambiguity in this work. We do not propose that adding integrity on top of an authenticated network is the sole method of securing a trusted repeater network; the combination of confidentiality, integrity and authenticity within a single protocol may offer a more effective approach to

reducing the assumptions on the network.

4 Future Directions

We outline three future directions which emerge from this work. First, as we recall from [Section 2.1.2](#) that an external difference family would give a weakly secure AMD code with minimal tag size, and as weak robustness is sufficient when relaying keys, a natural future direction is to search for an external difference family which satisfies the parameter constraints detailed in [Section 2.1.2](#). Second, as we recall from [Section A](#) that the SECOQC protocol aimed unsuccessfully at being able to identify cheaters, a natural future direction would be to create a protocol with this property. Third, as noted in [Section 3.2.2](#), our assumption that the network is static severely restricts its practicality. Rather than proposing refinements to our protocol within the current static model, we conclude with a broader open problem: the development of a trusted repeater model capable of representing dynamic networks.

References

- [BB84] Charles H. Bennett and Gilles Brassard. Quantum Cryptography: Public Key Distribution and Coin Tossing. In *International Conference on Computers, Systems & Singal Processing*, volume 1, pages 175–179, Bangalore, India, December 1984. DOI: [10.1016/j.tcs.2014.05.025](https://doi.org/10.1016/j.tcs.2014.05.025).
- [BHS09] Travis R. Beals, Kevin P. Hynes, and Barry C. Sanders. Distributed Authentication for Randomly Compromised Networks. *New Journal of Physics*, 11(8):085005, August 2009. ISSN: 1367-2630. DOI: [10.1088/1367-2630/11/8/085005](https://doi.org/10.1088/1367-2630/11/8/085005).
- [BR07] Mihir Bellare and Phillip Rogaway. Robust Computational Secret Sharing and a Unified Account of Classical Secret-Sharing Goals. In *Proceedings of the 14th ACM Conference on Computer and Communications Security, CCS '07*, pages 172–184, New York, NY, USA. Association for Computing Machinery, October 28, 2007. ISBN: 978-1-59593-703-2. DOI: [10.1145/1315245.1315268](https://doi.org/10.1145/1315245.1315268).
- [BS08] Travis R. Beals and Barry C. Sanders. Distributed Relay Protocol for Probabilistic Information-Theoretic Security in a Randomly-Compromised Network. In Reihaneh Safavi-Naini, editor, *Information Theoretic Security*, pages 29–39, Berlin, Heidelberg. Springer, 2008. ISBN: 978-3-540-85093-9. DOI: [10.1007/978-3-540-85093-9_4](https://doi.org/10.1007/978-3-540-85093-9_4).
- [CDF⁺08] Ronald Cramer, Yevgeniy Dodis, Serge Fehr, Carles Padró, and Daniel Wichs. Detection of Algebraic Manipulation with Applications to Robust Secret Sharing and Fuzzy Extractors. In Nigel Smart, editor, *Advances in Cryptology – EUROCRYPT 2008*. Volume 4965, pages 471–488. Springer, Berlin, Heidelberg, 2008. ISBN: 978-3-540-78966-6. DOI: [10.1007/978-3-540-78967-3_27](https://doi.org/10.1007/978-3-540-78967-3_27).
- [CDV94] Marco Carpentieri, Alfredo De Santis, and Ugo Vaccaro. Size of Shares and Probability of Cheating in Threshold Schemes. In Tor Helleseth, editor, *Advances in Cryptology – EUROCRYPT '93*, pages 118–125, Berlin, Heidelberg. Springer, 1994. ISBN: 978-3-540-48285-7. DOI: [10.1007/3-540-48285-7_10](https://doi.org/10.1007/3-540-48285-7_10).
- [MS81] R. J. McEliece and D. V. Sarwate. On sharing secrets and Reed-Solomon codes. *Commun. ACM*, 24(9):583–584, September 1981. ISSN: 0001-0782. DOI: [10.1145/358746.358762](https://doi.org/10.1145/358746.358762).

- [OK96] Wakaha Ogata and Kaoru Kurosawa. Optimum Secret Sharing Scheme Secure against Cheating. In Ueli Maurer, editor, *Advances in Cryptology — EUROCRYPT '96*, pages 200–211, Berlin, Heidelberg. Springer, 1996. ISBN: 978-3-540-68339-1. DOI: [10.1007/3-540-68339-9_18](https://doi.org/10.1007/3-540-68339-9_18).
- [Ren05] Renato Renner. *Security of Quantum Key Distribution*, Swiss Federal Institute of Technology Zürich, Zürich, 2005. 148 pages. arXiv: [quant-ph/0512258v2](https://arxiv.org/abs/quant-ph/0512258v2).
- [RK12] Stefan Rass and Sandra König. Turning Quantum Cryptography against itself: How to avoid indirect eavesdropping in quantum networks by passive and active adversaries. *International Journal on Advances in Systems and Measurements*, 5:22–33, 1 & 2, 2012. ISSN: 1942-261x. URL: <https://www.academia.edu/74435593/>.
- [Sha79] Adi Shamir. How to Share a Secret. *Commun. ACM*, 22(11):612–613, November 1, 1979. ISSN: 0001-0782. DOI: [10.1145/359168.359176](https://doi.org/10.1145/359168.359176).
- [Sin38] James Singer. A theorem in finite projective geometry and some applications to number theory. *Transactions of the American Mathematical Society*, 43(3):377–385, 1938. ISSN: 0002-9947. DOI: [10.2307/1990067](https://doi.org/10.2307/1990067).
- [SPD⁺10] Louis Salvail, Momtchil Peev, Eleni Diamanti, Romain Alléaume, Norbert Lütkenhaus, and Thomas Länger. Security of Trusted Repeater Quantum Key Distribution Networks. *Journal of Computer Security*, 18(1):61–87, January 1, 2010. ISSN: 0926-227X. DOI: [10.3233/JCS-2010-0373](https://doi.org/10.3233/JCS-2010-0373).
- [TW89] Martin Tompa and Heather Woll. How To Share a Secret with Cheaters. *Journal of Cryptology*, 1(3):133–138, October 1, 1989. ISSN: 1432-1378. DOI: [10.1007/BF02252871](https://doi.org/10.1007/BF02252871).
- [WC81] Mark N. Wegman and J. Lawrence Carter. New Hash Functions and Their Use in Authentication and Set Equality. *Journal of Computer and System Sciences*, 22(3):265–279, June 1981. ISSN: 0022-0000. DOI: [10.1016/0022-0000\(81\)90033-7](https://doi.org/10.1016/0022-0000(81)90033-7).

A Attack on the SECOQC Protocol

We summarise the SECOQC protocol illustrated in Figure 4. The protocol is executed after a secret-sharing scheme, and the core concept is the generation of a random parity check matrix Λ , with the subsequent computation of parity bits r over the secret. A two-time MAC key κ is taken from the secret-sharing scheme output, and tags are attached to subsequent messages to prevent adversarial modification.² The authors claim that this protocol can both detect and *identify* the malicious paths by inspecting from which path they receive invalid parity bits or tags.

As previously noted, the issue with this approach is that MACs only guarantee security when used with shared *identical* keys $\kappa = \kappa'$, and using the equality of values as keys in a MAC scheme constitutes circular reasoning and does not yield provable security. We now present a successful attack against the claimed identification property of the protocol. With this attack, the adversary is able to cause honest paths to be misidentified as malicious, while their own corrupted path is misidentified as honest.

We let the MAC be implemented with a Wegman-Carter construction [WC81]. Let $\mathcal{F}$ be an almost XOR-universal class of functions. Split the MAC key in three parts $\kappa_1 \parallel \kappa_2 \parallel \kappa_3 := \kappa$, where κ_1 identifies a function $f_{\kappa_1} \in \mathcal{F}$ and κ_2 and κ_3 will act as one-time pads. Alice computes her tag as

$$T := \text{MAC}_{\kappa}(\Lambda \| r) = f_{\kappa_1}(\Lambda \| r) \oplus \kappa_2. \quad (3)$$

²A two-time MAC key could be the concatenation of two one-time MAC keys or, in the case of Wegman-Carter MACs, a function index and two one-time pads.

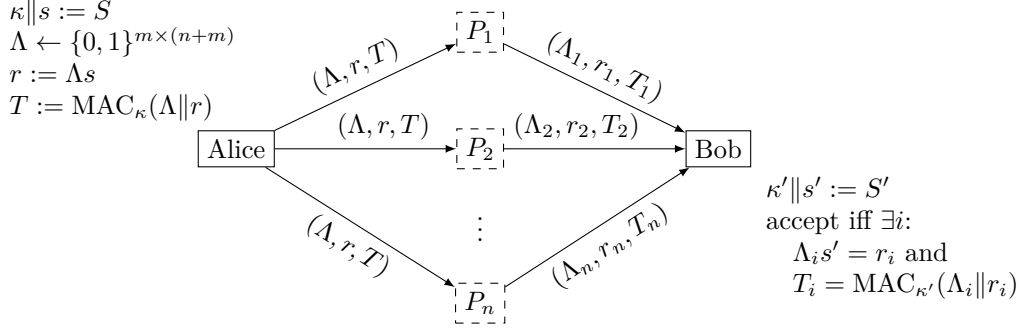


Figure 4: SECOQC integrity protocol [CDF⁺08]. This depicts step (2), the full protocol runs as follows: (1) Alice and Bob run a (non-robust) additive secret-sharing scheme to get S and S' respectively. (2) They run the depicted protocol so Bob can verify whether $S' = S$. (3) Bob replies with $(b, \text{MAC}_{\kappa}(b))$, where b is the accept/reject bit. (4) Alice and Bob deterministically select some bits from s to remove in order to correct for the information leaked via (Λ, r) .

Consider an adversary against the SECOQC protocol that controls a node in path P_n . Let this adversary select some algebraic shift Δ_2 and manipulate step (1) so that Alice gets share

$$S_n = \kappa_{1n} \| \kappa_{2n} \| \kappa_{3n} \| s_n \quad (4)$$

and Bob gets share

$$S'_n = \kappa_{1n} \| (\kappa_{2n} \oplus \Delta_2) \| \kappa_{3n} \| s_n. \quad (5)$$

When combined with the other shares, Alice computes MAC key $\kappa = \kappa_1 \| \kappa_2 \| \kappa_3$ and Bob computes $\kappa' = \kappa_1 \| (\kappa_2 \oplus \Delta_2) \| \kappa_3$. In step (2), Alice computes T with Equation (3) and sends T on every path. On path P_n , the adversary adds Δ_2 to the received tag, so that Bob receives

$$T_n = T \oplus \Delta_2, \quad (6)$$

while other paths forward the tag unmodified: $T_i = T$ for $i < n$.³ When Bob verifies T_n with key κ' , he accepts the tag because

$$T_n = T \oplus \Delta_2 = f_{\kappa_1}(\Lambda \| r) \oplus \kappa_2 \oplus \Delta_2 = \text{MAC}_{\kappa'}(\Lambda \| r). \quad (7)$$

Since $s = s'$ and the adversary did not change Λ and r , Bob also finds that the parity information is correct, so he accepts. At the same time, Bob finds that $T_i \neq T_n$ for all other paths, so he wrongly concludes that those paths must be dishonest.

B Formalisation of Security Definitions and Proofs

We adapt Bellare and Rogaway's formalisation of a secret-sharing scheme [BR07], in which confidentiality and robustness are modelled through distinct games. We do not modify their definition of confidentiality apart from minor notational adjustments. For robustness, we modify the existing model by incorporating explicit cheater detection and allowing the adversary to shift uncorrupted shares arbitrarily. We assume that the number of shares and the access structure are fixed parameters of the secret-sharing scheme. We will consider confidentiality and integrity through two separate games, IND-RELAY and FORGE-RELAY.

³If the adversary controls other paths, they can also deliver T_n on those paths.

We formally define the security using established game-based notions, where the probability that the adversary breaks security is defined as the probability of them winning the associated game. A hybrid proof is a sequence of hops between games such that there are only minor changes between successive games, so that we can bound the difference between the winning probabilities of subsequent games. The initial game describes the protocol and the final game models a game with a known bound on the winning probability.

One aspect not covered by the separate modelling of confidentiality and integrity is the potential interplay between the two properties. For instance, if the adversary can see the accept/reject bit, this could inadvertently reveal information about the secret. We leave the development of an appropriate extension of the model which addresses this limitation for future work.

B.1 Confidentiality

The formal indistinguishability game IND-SSS for a secret-sharing scheme Π_s is given in Listing 2. An adversary $\mathcal{A}$ provides two secrets (subroutine $\mathcal{A}_0$), the game selects one at random and executes **Share**. The adversary can corrupt any unqualified set of shares, tracked by the game in a set T (Listing 2), and then guesses which share the game picked (subroutine $\mathcal{A}_1$). The game outputs one (denoted with the indicator function $\llbracket \cdot \rrbracket$) if the adversary is unqualified and guessed correct. A secret-sharing scheme Π_s has *perfect* confidentiality if for all $\mathcal{A}$ we have

$$\Pr[\text{IND-SSS}(\Pi_s, \mathcal{A}) = 1] = 1/2, \quad (8)$$

in other words, the adversary cannot do better than guessing randomly.

Listing 2 Indistinguishability game for secret-sharing scheme $\Pi_s = (\text{Share}, \text{Recover})$ [BR07].

```

1: game IND-SSS( $\Pi_s, \mathcal{A}$ )
2:    $b \xleftarrow{\$} \{0, 1\}$  ▷ Challenge bit  $b$ 
3:    $s_0, s_1 \leftarrow \mathcal{A}_0()$  ▷ Adversary chooses secrets  $s_0, s_1$ 
4:    $\vec{S} \xleftarrow{\$} \text{Share}(s_b)$  ▷  $\vec{S} = (S_1, \dots, S_n)$ 
5:    $b' \leftarrow \mathcal{A}_1^{\text{SSS.CORRUPT}(\cdot)}()$  ▷ Adversary guesses  $b'$ 
6:   return  $\llbracket b = b' \text{ and } T \text{ is unqualified} \rrbracket$ 
7: oracle SSS.CORRUPT( $i$ )
8:    $T \leftarrow T \cup \{i\}$  ▷ Initially  $T = \emptyset$ 
9:   return  $S_i$ 
```

In Listing 3, we define the indistinguishability game for trusted repeater networks. The main differences with IND-SSS are that corruption reveals the relay keys and that the adversary gets full control over the relaying by deciding which (possibly forged) ciphertext to deliver to which node and in which order. For simplicity, the game is defined for a protocol Π_r that uses secret sharing and simple relaying, but it can be generalised by extracting the protocol-specific code into subroutines. Note that R (Listing 3) is a set containing information about which nodes have already relayed; $(i, j) \in R$ means that key q_{ij} was already deleted because $\text{RELAY}(i, j, \cdot)$ was called before.

Theorem 3 (Confidentiality). *Let G be a trusted-repeater network with n paths, each of length at most ℓ , let the QKD keys be ε -private against quantum side information according to Definition 3, and let Π_r be the protocol of Listing 1. Then, for all (computationally unbounded) adversaries $\mathcal{A}$:*

$$|\Pr[\text{IND-RELAY}(\Pi_r, \mathcal{A}) = 1] - 1/2| \leq n\ell\varepsilon.$$

Listing 3 Indistinguishability game for multipath relay protocol Π_r using the secret-sharing scheme (Share, Recover) over a trusted repeater network.

```

1: game IND-RELAY( $\Pi_r, \mathcal{A}$ )
2:    $b \xleftarrow{\$} \{0, 1\}$ 
3:    $s_0, s_1 \leftarrow \mathcal{A}_0()$ 
4:    $\vec{S} \xleftarrow{\$} \text{Share}(s_b)$   $\triangleright \vec{S} = (S_1, \dots, S_n)$ 
5:   for  $1 \leq i \leq n$  and  $1 \leq j \leq \ell$  do
6:      $q_{ij} \xleftarrow{\$} \text{QKD}$   $\triangleright$  QKD generated keys as per Definition 3
7:     for  $1 \leq i \leq n$  do
8:        $c_{i1} = S_i + q_{i1}$   $\triangleright \vec{c} = (c_{11}, \dots, c_{n1})$ 
9:        $b' \leftarrow \mathcal{A}_1^{\text{RELAY}(\cdot, \cdot, \cdot), \text{R.CORRUPT}(\cdot, \cdot)}(\vec{c})$   $\triangleright \vec{c'} = (c'_{1\ell}, \dots, c'_{n\ell})$ 
10:    return  $\llbracket b = b' \text{ and } T \text{ is unqualified} \rrbracket$ 
11: oracle RELAY( $i, j, c'_{i(j-1)}$ )
12:   if  $(i, j) \in R$  then
13:     return  $\perp$   $\triangleright$  Relay is allowed at most once to model key deletion
14:    $R \leftarrow R \cup \{(i, j)\}$   $\triangleright$  Initially  $R = \emptyset$ 
15:   return  $c'_{i(j-1)} - q_{i(j-1)} + q_{ij}$   $\triangleright$  Re-encrypt
16: oracle R.CORRUPT( $i, j$ )
17:   if  $(i, j) \in R$  then
18:     return  $\perp$   $\triangleright$  Keys were already deleted after relaying
19:    $T \leftarrow T \cup \{i\}$ 
20:   return  $(q_{i(j-1)}, q_{ij})$ 

```

Proof. Recall the definition of ε -privacy against quantum side information (Definition 3). We give the proof as two game hops. We define game G_1 as equal to IND-RELAY (Listing 3), except that we replace Listing 3 with the following so that G_1 samples all keys uniformly random: $q_{ij} \xleftarrow{\$} \mathcal{G}$. Since individual QKD keys are ε -private according to Definition 3, and there are $n\ell$ keys in total, it follows from Lemma 1 that

$$|\Pr[\text{IND-RELAY}(\Pi_r, \mathcal{A}) = 1] - \Pr[G_1 = 1]| \leq n\ell\varepsilon. \quad (9)$$

We now show that G_1 is secure by giving a reduction $\mathcal{B}$ to the IND-SSS game, detailed in Listing 4. While the reduction involves detailed steps, its structure is conceptually straightforward. Before corruption, relaying returns random ciphertexts to simulate re-encryption with a random key (Listing 4). Upon corruption of path P_i , $\mathcal{B}$ corrupts in the secret sharing game to get the share (Listing 4) and computes all the keys in the path. For any ciphertext seen by the adversary $\mathcal{A}$ of game G_1 , the key is chosen so as to produce a value consistent with the adversary's view (Listing 4), for example $q_{i1} = c_{i1} - S_i$. Keys for unseen ciphertexts are sampled uniformly random (Listing 4). After corruption, all relays are computed with the derived keys (Listing 4). Finally, $\mathcal{B}_1$ outputs whatever $\mathcal{A}_1$ outputs (Listing 4). Since $\mathcal{B}$ wins IND-SSS exactly when $\mathcal{A}$ wins G_1 , we have

$$\Pr[G_1 = 1] = \Pr[\text{IND-SSS}(\Pi_s, \mathcal{B}) = 1]. \quad (10)$$

Since the secret-sharing scheme is perfectly secure (Equation (8)), we can combine Equations (8) to (10) to conclude

$$|\Pr[\text{IND-RELAY}(\Pi_r, \mathcal{A}) = 1] - 1/2| \leq n\ell\varepsilon. \quad \square$$

Listing 4 IND-SSS adversary $\mathcal{B} = (\mathcal{B}_0, \mathcal{B}_1)$, using G_1 adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$.

```

1: procedure  $\mathcal{B}_0()$ 
2:   return  $\mathcal{A}_0()$ 
3: procedure  $\mathcal{B}_1()$ 
4:    $\vec{c} \xleftarrow{\$} \mathcal{G}^n$  ▷ Initialise with random ciphertexts
5:   return  $\mathcal{A}_1^{\text{RELAY}(\cdot, \cdot, \cdot), \text{R.CORRUPT}(\cdot, \cdot)}(\vec{c})$ 
6: oracle  $\text{RELAY}(i, j, c'_{i(j-1)})$ 
7:   if  $(i, j) \in R$  then
8:     return  $\perp$ 
9:    $R \leftarrow R \cup \{(i, j)\}$ 
10:  if  $i \in T$  then
11:     $c_{ij} \xleftarrow{\$} \mathcal{G}$  ▷ Return random ciphertext on uncorrupted paths
12:  else
13:     $c_{ij} \leftarrow c'_{i(j-1)} - q_{i(j-1)} + q_{ij}$  ▷ Re-encrypt with keys computed at corruption
14:  return  $c_{ij}$ 
15: oracle  $\text{R.CORRUPT}(i, j)$ 
16:  if  $(i, j) \in R$  then
17:    return  $\perp$ 
18:  if  $i \notin T$  then
19:     $T \leftarrow T \cup \{i\}$ 
20:     $S_i \leftarrow \text{SSS.CORRUPT}(i)$ 
21:     $q_{i1} \leftarrow c_{i1} - S_i$  ▷ Set to value consistent with adversary view (first step)
22:    for  $2 \leq j' \leq \ell$  do
23:      if  $(i, j') \in R$  then ▷ Note that  $c_{ij'} \neq \perp$  and  $c'_{i(j'-1)} \neq \perp$ 
24:         $q_{ij'} \leftarrow c_{ij'} - c'_{i(j'-1)} + q_{i(j'-1)}$  ▷ Consistency with adversary view
25:      else
26:         $q_{ij'} \xleftarrow{\$} \mathcal{G}$  ▷ Other keys sampled from random
27:  return  $(q_{i(j-1)}, q_{ij})$ 

```

B.2 Integrity

We define shift-robustness for a secret-sharing scheme where shares S_i are elements of some additive group $\mathcal{G}$. Formally, this is modelled by the game SHIFT-ROBUST-SSS defined in Listing 5. An unqualified adversary can select the initial secret, corrupt shares to learn and change them, and shift uncorrupted shares by adding any chosen group element, all with the goal of having the game accept a secret different than the one initially provided. A secret-sharing scheme $\Pi_s = (\text{Share}, \text{Recover})$ is said to be δ -shift-robust if for all adversaries $\mathcal{A}$ we have

$$\Pr[\text{SHIFT-ROBUST-SSS}(\Pi_s, \mathcal{A}) = 1] \leq \delta. \quad (11)$$

Listing 5 Shift-Robustness game for secret-sharing scheme $\Pi_s = (\text{Share}, \text{Recover})$, adapted from [BR07].

```

1: game SHIFT-ROBUST-SSS( $\Pi_s, \mathcal{A}$ )
2:    $s \leftarrow \mathcal{A}_0()$  ▷ Adversary chooses secret  $s$ 
3:    $\vec{S} \xleftarrow{\$} \text{Share}(s)$ 
4:    $\vec{a} \leftarrow \mathcal{A}_1^{\text{SSS.CORRUPT}(\cdot)}()$  ▷ Adversary chooses shares and shifts:  $\vec{a} = (a_1, \dots, a_n)$ 
5:   for  $1 \leq i \leq n$  do
6:     if  $i \in T$  then
7:        $S'_i \leftarrow a_i$  ▷ Corrupted share replaced by  $a_i$ 
8:     else
9:        $S'_i \leftarrow S_i + a_i$  ▷ Add  $a_i$  to uncorrupted share
10:  return  $\llbracket \text{Recover}(\vec{S}') \notin \{s, \perp\} \text{ and } T \text{ is unqualified} \rrbracket$  ▷  $\vec{S}' = (S'_1, \dots, S'_n)$ 
11: oracle SSS.CORRUPT( $i$ )
12:    $T = T \cup \{i\}$ 
13:  return  $S_i$ 

```

By Lemma 2, we know that Construction 2, namely $(\text{Share}^*, \text{Recover}^*)$ instantiated with a δ -robust AMD-encoding, is δ -shift-robust.

In Listing 6 we define the game for modelling the integrity of multipath relaying protocols in the trusted repeater network context. Similar to the IND-RELAY game, we give the adversary the power to corrupt nodes and to control relaying. In addition, we let the adversary choose c' , which is then decrypted and recovered by the game.

Theorem 4 (Integrity). *Let G be a trusted-repeater network with n paths, each of length at most ℓ , let the QKD keys be ε -private against quantum side information according to Definition 3, and let Π_r be the protocol of Listing 1. Then, for all (computationally unbounded) adversaries $\mathcal{A}$:*

$$\Pr[\text{FORGE-RELAY}(\Pi_r, \mathcal{A}) = 1] \leq n\ell\varepsilon + \delta.$$

Proof. We define game G_2 as equal to FORGE-RELAY (Listing 6), except that we replace Listing 6 with the following so that all QKD keys are sampled as uniformly random group elements: $q_{ij} \xleftarrow{\$} \mathcal{G}$. Since individual QKD keys are ε -private against quantum side information (Definition 3) and there are $n\ell$ keys, it follows from Lemma 1 that

$$|\Pr[\text{FORGE-RELAY}(\Pi_r, \mathcal{A}) = 1] - \Pr[G_2 = 1]| \leq n\ell\varepsilon. \quad (12)$$

We now show that G_2 is secure by giving a reduction $\mathcal{B}$, detailed in Listing 7: $\mathcal{B}$ is a SHIFT-ROBUST-SSS adversary and simulates game G_2 against black-box adversary $\mathcal{A}$. Note that the inputs to $\mathcal{A}$ are identical to that of Listing 4: $\vec{c}$ is sampled uniformly random and the simulation of oracles RELAY and R.CORRUPT are identical. However, when $\mathcal{A}_1$

Listing 6 Integrity game for relay protocol Π_r , using the secret-sharing scheme (Share, Recover) over a trusted repeater network.

```

1: game FORGE-RELAY( $\Pi_r, \mathcal{A}$ )
2:    $s \leftarrow \mathcal{A}_0()$ 
3:    $\vec{S} \leftarrow \text{Share}(s)$ 
4:   for  $1 \leq i \leq n$  and  $1 \leq j \leq \ell$  do
5:      $q_{ij} \xleftarrow{\$} \text{QKD}$  ▷ QKD generated keys as per Definition 3
6:     for  $0 \leq i < n$  do
7:        $c_{i1} = S_i + q_{i1}$  ▷  $\vec{c} = (c_{11}, \dots, c_{n1})$ 
8:        $\vec{c}' \leftarrow \mathcal{A}_1^{\text{RELAY}(\cdot, \cdot, \cdot), \text{R.CORRUPT}(\cdot, \cdot)}(\vec{c})$  ▷  $\vec{c}' = (c'_{1\ell}, \dots, c'_{n\ell})$ 
9:       for  $1 \leq i \leq n$  do
10:         $S'_i \leftarrow c'_{i\ell} - q_{i\ell}$ 
11:      return  $\llbracket \text{Recover}(\vec{S}') \notin \{s, \perp\} \text{ and } T \text{ is unqualified} \rrbracket$  ▷  $\vec{S}' = (S'_1, \dots, S'_n)$ 
12: oracle RELAY( $i, j, c'_{i(j-1)}$ )
13:   if  $(i, j) \in R$  then
14:     return  $\perp$  ▷ Relay at most once, to model deletion of keys
15:    $R \leftarrow R \cup \{(i, j)\}$  ▷ Initially  $R = \emptyset$ 
16:   return  $c_{ij} \leftarrow c'_{i(j-1)} - q_{i(j-1)} + q_{ij}$ 
17: oracle R.CORRUPT( $i, j$ )
18:   if  $(i, j) \in R$  then
19:     return  $\perp$  ▷ Keys were already deleted after relaying
20:    $T \leftarrow T \cup \{i\}$ 
21:   return  $(q_{i(j-1)}, q_{ij})$ 

```

returns a vector $\vec{c}'$ on Listing 7, $\mathcal{B}_1$ needs to translate this into a vector $\vec{a}$ for the SHIFT-ROBUST-SSS challenger (Listing 7), depending on the status of relaying and corruption per path. First, missing ciphertexts are translated into missing shifts (Listing 7). Second, ciphertexts on corrupted paths can simply be decrypted (Listing 7). Third, incomplete relays on uncorrupted paths will be decrypted with a unused uniformly random key in G_2 , resulting in a uniformly random shift (Listing 7). Finally, in a complete end-to-end relay, $\mathcal{B}_1$ computes the total shift (Listing 7) as the sum over the path of differences between delivered and sent ciphertexts: $a_i = \sum_{j=1}^{\ell} (c'_{ij} - c_{ij})$.

For the full relay on an uncorrupted path in G_2 , we have $c_{i1} = S_i + q_{i1}$ and $c_{ij} = c'_{i(j-1)} - q_{i(j-1)} + q_{ij}$ for $2 \leq j \leq \ell$. Therefore we see that the simulated shift a_i indeed encodes the end-to-end difference between the sent and received share:

$$\begin{aligned}
S_i + a_i &= S_i + \sum_{j=1}^{\ell} (c'_{ij} - c_{ij}) \\
&= S_i + c'_{i1} - c_{i1} + \sum_{j=2}^{\ell} (c'_{ij} - c_{ij}) \\
&= S_i + c'_{i1} - (S_i + q_{i1}) + \sum_{j=2}^{\ell} (c'_{ij} - (c'_{i(j-1)} - q_{i(j-1)} + q_{ij})) \\
&= c'_{i\ell} - q_{i\ell} = S'_i,
\end{aligned}$$

where the fourth equality follows since we have a telescoping sum and the final equality depicts decryption in G_2 .

Listing 7 IND-SSS adversary $\mathcal{B} = (\mathcal{B}_0, \mathcal{B}_1)$, using G_2 adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$.

```

1: procedure  $\mathcal{B}_0()$ 
2:   return  $\mathcal{A}_0()$ 
3: procedure  $\mathcal{B}_1()$ 
4:    $\vec{c} \xleftarrow{\$} \mathcal{G}^n$  ▷  $\vec{c} = (c_{11}, \dots, c_{n1})$ 
5:    $\vec{c'} \leftarrow \mathcal{A}_1^{\text{RELAY}(\cdot, \cdot, \cdot), \text{R.CORRUPT}(\cdot, \cdot)}(\vec{c})$  ▷  $\vec{c'} = (c'_{1\ell}, \dots, c'_{n\ell})$ 
6:   for  $1 \leq i \leq n$  do
7:     if  $c'_{i\ell} = \perp$  then ▷ No relay message delivered
8:        $a_i \leftarrow \perp$ 
9:     else if  $q_{i\ell} \neq \perp$  then ▷  $\text{R.CORRUPT}(i)$  was called
10:       $a_i \leftarrow c'_{i\ell} - q_{i\ell}$ 
11:     else if  $\exists j : 1 \leq j < \ell$  and  $c'_{ij} = \perp$  then ▷ Not fully relayed
12:        $a_i \xleftarrow{\$} \mathcal{G}$ 
13:     else ▷ Fully relayed
14:        $a_i \leftarrow \sum_{j=1}^{\ell} (c'_{ij} - c_{ij})$ 
15:   return  $\vec{a}$  ▷  $\vec{a} = (a_0, \dots, a_{n-1})$ 
16: oracle  $\text{RELAY}(i, j, c'_{i(j-1)}) \dots$  ▷ See Listing 4, Listing 4
17: oracle  $\text{R.CORRUPT}(i, j) \dots$  ▷ See Listing 4, Listing 4

```

Then $\mathcal{B}$ wins the SHIFT-ROBUST-SSS game exactly when $\mathcal{A}$ would win game G_2 . From this we conclude

$$\Pr[G_2 = 1] = \Pr[\text{SHIFT-ROBUST-SSS}(\Pi_s, \mathcal{B}) = 1]. \quad (13)$$

By Equation (11), we know that this probability is bounded by δ . Combining Equations (11) to (13), we get

$$\Pr[\text{FORGE-RELAY}(\Pi_r, \mathcal{A}) = 1] \leq n\ell\varepsilon + \delta. \quad \square$$

C Implementation

The script in Listing 8 performs secret sharing using the additive scheme and AMD encoding. It is provided with the intention of assisting engineers in the practical deployment of robust secret sharing within trusted repeater networks in QKD systems.

Listing 8: SageMath proof of concept implementation of AMD encoded additive secret sharing. See <https://github.com/sebastianv89/AMD-secret-sharing> for a version including unit-tests and dynamic parameter selection.

```

"""WARNING: PROOF OF CONCEPT, DO NO USE FOR SECURING REAL DATA!"""

# System parameters (example)
d = 3
F = GF(2^86)

def share(n, secret):
    """Return n shares, shared via additive sharing."""
    shares = [[F.random_element() for _ in range(len(secret))]
               for _ in range(n-1)]
    shares.append([secret[i] - sum(share[i] for share in shares)
                   for i in range(len(secret))])
    return shares

def recover(shares):
    """Recover the secret from the additive shares."""
    return [sum(share[i] for share in shares)
            for i in range(len(shares[0]))]

def tag(x, s):
    """Compute f(x, s), using Horner's method."""
    res = x
    for coef in s:
        res = x*res + coef # IMPORTANT: multiplication in GF(q)
    return x*res

def amd_encode(s):
    """Return (s, x, f(x, s)): the systematic encoding of s."""
    x = F.random_element()
    return s + [x, tag(x, s)]

def amd_decode(encoded):
    """Return s or None: the decoding of (s, x, y)."""
    *s, x, y = encoded
    if tag(x, s) == y:
        return s
    return None

def amd_share(n, secret):
    """Return n shares, shared via AMD encoded additive sharing."""
    return share(n, amd_encode(secret))

def amd_recover(shares):
    """Recover the secret from the AMD encoded additive shares."""
    return amd_decode(recover(shares))

```